```
template<typename D>
LIST<D>::LIST() {  H = T = nullptr;  elem = 0;  }

template<typename D>
LIST<D>::~LIST() {
   while(H) {
      node<D>* tmp = H; H = H->P;
      delete tmp;
   }
   T = nullptr; elem = 0;
}

template<typename D>
node<D>* LIST<D>::pointer(const long int index) const{
   index = index<0? elem+index : index;

   if(index>=elem || index<0) return nullptr;
   node<D> *iter=H;
   for(long int i=0; i<index; i++) iter=iter->P;
   return iter;
}

template<typename D>
D& LIST<D>::operator[](const long int index) {
   if(H=T=nullptr) throw std::runtime_error("Tree is empty");

   node<D>* ptr = pointer(index);    // get the node pointer
   if(!ptr) throw std::out_of_range("Index out of bounds");
   return ptr->K;                    // return reference to the value
}

template<typename D>
const D& LIST<D>::operator[](const long int index) const {
   if(H=T=nullptr) throw std::runtime_error("Tree is empty");

   const node<D>* ptr = pointer(index);    // get the node pointer
   if(!ptr) throw std::out_of_range("Index out of bounds");
   return ptr->K;                          // return const reference
}
```

🧠 **Copy Semantics {Duplicate data}**

✅ **1. Copy Constructor → NEW object + copy**

```
template<typename D>
LIST<D>::LIST(const LIST<D>& other) {
  H = T = nullptr; elem = 0;
  node<D>* iter = other.H;
  while(iter){
    insert(iter->K); iter = iter->P;
  }
}

LIST<int> a;
LIST<int> b = a;    // 🔥 Copy Constructor
LIST<int> c(a);     // 🔥 Copy Constructor
// b and c are being created Source is an lvalue (a)
```

✅ **2. Copy Assignment Operator → EXISTING object + copy**

```
template<typename D>
LIST<D>& LIST<D>::operator=(const LIST<D>& other){
  if(this != &other){
    clear(H);
    H=T=nullptr; elem=0;

    node<D>* iter=other.H;
    while(iter){
      insert(iter->K); iter=iter->P;
    }
  }
  return *this;
}

LIST<int> a;
LIST<int> b;
b = a;  // 🔥 Copy Assignment Operator
// b already exists a is an lvalue
```

🧠 **Move Semantics (C++11) {Transfer data (faster than copy)}**

```
int a = 5;

int &x = a;     // ✅ OK (lvalue)
int &y = 10;    // ❌ ERROR
const int &m = a;   // ✅ OK, lvalue
const int &n = 20;  // ✅ OK, rvalue

int &&z = 10;   // ✅ OK (rvalue)
int &&w = a;    // ❌ ERROR
const int &&o = 10;   // ✅ OK
const int &&p = a     // ❌ ERROR

void fun(int &x)  { std::cout << "Lvalue\n"; }
void fun(int &&x) { std::cout << "Rvalue\n"; }

fun(a);    // Lvalue
fun(10);   // Rvalue
```

✅ **1. Move Constructor → when creating a NEW object 👉 Used during initialization**

```
template<typename D>
LIST<D>::LIST(LIST<D>&& other) noexcept {
   H = other.H;
   T = other.T;
   elem = other.elem;

   // Leave other in safe empty state
   other.H = other.T = nullptr;
   other.elem = 0;
}
LIST<int> a;
LIST<int> b = std::move(a);   // 🔥 Move Constructor
LIST<int> c(std::move(a));    // 🔥 Move Constructor
LIST<int> create(){
   LIST<int> temp;
   return temp;   // move constructor (or RVO)
}
LIST<int> d = create();  // 🔥 Move Constructor
```

✅ **2. Move Assignment → when object already exists** 👉 **Used during assignment**

```
template<typename D>
LIST<D>& LIST<D>::operator=(LIST<D>&& other) noexcept {
  if(this != &other){
    while(H){
      node<D>* tmp = H;
      H = H->P;
      delete tmp;
    }
    H = other.H;
    T = other.T;
    elem = other.elem;

    other.H = other.T = nullptr;
    other.elem = 0;
  }
  return *this;
}
LIST<int> a;
LIST<int> b;
b = std::move(a);  // 🔥 Move Assignment Operator
```

🧠 **RVO / NRVO (Optimization) {Direct construction}**

✅ Case A: **RVO [Return Value Optimization] (most common)**
"When we return a temporary like return LIST<int>();, it directly creates object in caller"
👉 And since C++17, this is guaranteed (no copy/move at all)
👉 Best case (no overhead at all)

```
LIST<int> create(){
   return LIST<int>();   // temporary (unnamed) object
}
LIST<int> x = create();   // Compiler directly creates it inside x
```

❌ No copy
❌ No move

✅ Case B: **NRVO [Named Return Value Optimization]**
"We return an object created inside the function"
👉 NRVO is not guaranteed (no copy/move)

```
LIST<int> create(){
   LIST<int> temp;    // NRVO candidate
   return temp;       // temp is a named variable
}
LIST<int> x = create();  // Compiler may construct it directly into x
```

❌ No copy
❌ No move

✅ Case C: Multiple return variables → ❌ NRVO fails

```
LIST<int> create(bool flag){
   LIST<int> a, b;
   if(flag) return a;
   else return b;
}
```

Two different variables: a and b Compiler cannot decide one fixed object
👉 ❌ NRVO does NOT apply 👉 ✅ Move constructor is used
👉 "NRVO cannot be applied because the compiler cannot determine a single return object at compile time due to multiple possible return variables."